Web 2 – Séance 12

Événements et formulaires

Événements : DOM et React

- Frontend old school : on écoute les événements du DOM avec `addEventListener`
- React : on passe la fonction à l'élément JSX avec `onClick`, `onChange`, `onSubmit`…

```
// JavaScript "old school" (séance 06)
const button = document.querySelector("#add-button");
button.addEventListener("click", () ⇒ console.log("Cliqué !"));
```

```
// React
<button onClick={() ⇒ console.log("Cliqué !")}>Ajouter</button>
```

- Nom en **camelCase** : `onClick`, `onChange`, `onSubmit`, `onKeyDown`, `onMouseEnter`, …
- La valeur est une **fonction**
- Le listener est déclaré directement sur l'élément concerné dans le JSX, ou via une fonction handler

Passer une fonction, pas l'appeler

```jsx
const RecipeActions = () => {
  const handleDelete = () => {
    console.log("Suppression");
  };

  return (
    <>
      <button onClick={handleDelete}>Supprimer</button>              {/* ✅ fonction */}
      <button onClick={() => handleDelete()}>Supprimer</button>      {/* ✅ fonction fléchée */}
      <button onClick={handleDelete()}>Supprimer</button>            {/* ❌ appel au rendu */}
    </>
  );
};
```

- `onClick={handleDelete()}` exécute la fonction **pendant le rendu** et passe son résultat (`undefined`)
- Fonction fléchée : utile pour passer un argument, `onClick={() => handleDelete(recipe.id)}`
- Convention :
  - Événement `foo` (e.g. `click`)
  - Attribut JSX `onFoo` (e.g. `onClick`)
  - Handler `handleFoo` (e.g. `handleClick`)

Typer les événements

```tsx
import type { ChangeEvent, MouseEvent, SubmitEvent } from "react";

const handleSubmit = (event: SubmitEvent<HTMLFormElement>) => { // FormEvent trouvable en ligne, déprécié récemment
  event.preventDefault(); // pas de rechargement de la page
};

const handleChange = (event: ChangeEvent<HTMLInputElement>) => {
  console.log(event.target.value);  // valeur saisie, toujours une string
};

const handleClick = (event: MouseEvent<HTMLButtonElement>) => {
  console.log(event.currentTarget); // le bouton
};
```

- Sans fonction handler, le type peut être inféré : `onChange={(e) => ...}`
- Si on n'a pas besoin de l'événement, on peut l'omettre : `onClick={() => console.log("Cliqué !")}`
- Si le type de l'événement n'est pas nécessaire, on peut utiliser `SyntheticEvent` générique à la place : `handleClick = (event: SyntheticEvent) => { ... }`
- ⚠️ Les types viennent de `"react"` : sans l'import, `MouseEvent`, `KeyboardEvent` et `SubmitEvent` désignent les types **natifs** du DOM

Deux façons de gérer un champ

- **Non contrôlé** : le navigateur garde la valeur du champ, et on la lit à la demande

```
<form onSubmit={(e) => {
  e.preventDefault();
  const input = e.currentTarget.elements.namedItem("title") as HTMLInputElement;
  console.log(input.value);
}}>
  <input name="title" />
  <button type="submit">Créer</button>
</form>
```

Contrôlé : l'état React est la **seule source de vérité**, le champ affiche l'état.

```
const [title, setTitle] = useState("");
<input value={title} onChange={(e) => setTitle(e.target.value)} />
```

- `value={title}` : le champ affiche **toujours** la valeur de l'état
- `onChange` : à chaque frappe, on met l'état à jour, ce qui provoque un rendu et réaffiche le champ

Pourquoi contrôler un champ ?

La valeur étant dans l'état, elle est disponible **à tout moment** pendant le rendu :

```
const [title, setTitle] = useState("");

return (
  <>
    <input value={title} onChange={(e) ⇒ setTitle(e.target.value)} />
    <p>{title.length} / 100 caractères</p>        {/* affichage en direct */}
    <button disabled={title.trim() === ""}>Créer</button>  {/* bouton déduit */}
    <button onClick={() ⇒ setTitle("")}>Effacer</button>  {/* réinitialisation */}
  </>
);
```

- Validation en direct, compteur, aperçu : tout est **déduit** de l'état
- Réinitialiser le champ = remettre l'état à sa valeur initiale
- Même principe qu'en séance 11 : on change l'état, React met à jour l'affichage

Champs de différents types

```
// Texte long
<textarea value={description} onChange={(e) => setDescription(e.target.value)} />

// Liste déroulante : value sur le select, pas sur les options
<select value={categoryId} onChange={(e) => setCategoryId(e.target.value)}>
  <option value="1">Entrée</option>
  <option value="2">Plat</option>
</select>

// Case à cocher : checked au lieu de value
<input
  type="checkbox"
  checked={isVegetarian}
  onChange={(e) => setIsVegetarian(e.target.checked)}
/>
```

⚠️ `e.target.value` est **toujours une chaîne**, même pour `<input type="number">`.

```
<input type="number" value={prepTime} onChange={(e) => setPrepTime(e.target.value)} />
// prepTime vaut "15", pas 15 : la conversion Number(prepTime) se fait à la soumission
```

Soumettre un formulaire

```tsx
const TitleForm = () => {
  const [title, setTitle] = useState("");

  const handleSubmit = (event: SubmitEvent<HTMLFormElement>) => {
    event.preventDefault(); // pas de rechargement de la page
    console.log("Nouveau titre :", title);
    setTitle("");           // vider le champ
  };

  return (
    <form onSubmit={handleSubmit}>
      <input value={title} onChange={(e) => setTitle(e.target.value)} required />
      <button type="submit">Créer</button>
    </form>
  );
};
```

- `onSubmit` sur le `<form>` (et pas `onClick` sur le bouton) : fonctionne aussi avec la touche Entrée
- `preventDefault()` : même raison qu'en séance 06, le comportement par défaut recharge la page
- `required`, `min`, `max`, `type="email"` : validation HTML native, avant `onSubmit`

Un état objet pour tout le formulaire

Quand un formulaire a beaucoup de champs, on peut regrouper leurs valeurs dans un seul objet.

```
interface RecipeFormData {
  title: string;
  description: string;
  prepTime: string; // chaînes : ce que l'utilisateur a tapé
  cookTime: string;
  categoryId: string;
}

const [form, setForm] = useState<RecipeFormData>({
  title: "",
  description: "",
  prepTime: "",
  cookTime: "",
  categoryId: "",
});
```

- Les champs numériques restent des chaînes dans l'état du formulaire
- On convertit en nombre une seule fois, à la soumission
- Pour modifier un champ : **nouvel objet** avec spread operator, on ne peut pas modifier l'objet existant

Un seul handler pour tous les champs

```
const handleChange = (event: ChangeEvent<HTMLInputElement | HTMLTextAreaElement>) => {
  const { name, value } = event.target;
  setForm({ ...form, [name]: value });
};

<input name="title" value={form.title} onChange={handleChange} />
<input name="prepTime" type="number" value={form.prepTime} onChange={handleChange} />
<textarea name="description" value={form.description} onChange={handleChange} />
```

- L'attribut `name` de chaque champ correspond à une propriété de `form`
- `[name]: value` : **nom de propriété calculé** – la clé de l'objet est la valeur de la variable `name`
- `{ ...form, [name]: value }` : copie de `form` dont une seule propriété est remplacée

```
const name = "prepTime";
const copy = { ...form, [name]: "15" }; // équivaut à { ...form, prepTime: "15" }
```

Formulaires avec MUI

`TextField` regroupe label, champ et message d'aide ; il se contrôle exactement comme un `<input>`.

```
import { Button, MenuItem, Stack, TextField } from "@mui/material";

<Stack component="form" spacing={2} onSubmit={handleSubmit}>
  <TextField label="Titre" name="title" value={form.title} onChange={handleChange} required />
  <TextField
    label="Description" name="description" value={form.description}
    onChange={handleChange} multiline rows={3}
  />
  <TextField label="Préparation (min)" name="prepTime" type="number"
    value={form.prepTime} onChange={handleChange} />
  <TextField select label="Catégorie" name="categoryId"
    value={form.categoryId} onChange={handleChange}>
    <MenuItem value="1">Entrée</MenuItem>
    <MenuItem value="2">Plat</MenuItem>
  </TextField>
  <Button type="submit" variant="contained">Créer la recette</Button>
</Stack>
```

- `component="form"` : le `Stack` génère un `<form>` et accepte `onSubmit`
- `select` : le `TextField` devient une liste déroulante, les options sont des `MenuItem`

Valider les données

```
const [form, setForm] = useState<RecipeFormData>(initialForm);
const [submitted, setSubmitted] = useState(false);

const titleError = form.title.trim().length < 3;
const prepTimeError = form.prepTime === "" || Number(form.prepTime) < 0;
const isValid = !titleError && !prepTimeError;

const handleSubmit = (event: SubmitEvent<HTMLFormElement>) => {
  event.preventDefault();
  setSubmitted(true);
  if (!isValid) return;
  console.log({ ...form, prepTime: Number(form.prepTime) });
};

<TextField
  label="Titre" name="title" value={form.title} onChange={handleChange}
  error={submitted && titleError}
  helperText={submitted && titleError ? "Au moins 3 caractères" : ""}
/>
```

- `submitted` : n'afficher les erreurs qu'après une première tentative, pas dès l'ouverture du formulaire
- ⚠️ La validation du frontend améliore l'expérience, mais **ne remplace pas** celle du backend

Récapitulatif Séance 12

- **Événements React** – `onClick`, `onChange`, `onSubmit` en camelCase, valeur = fonction
- **Handler** – Passer la fonction, ne pas l'appeler : `onClick={handleClick}`
- **Typage** – `ChangeEvent<HTMLInputElement>`, `SubmitEvent<HTMLFormElement>` … importés depuis `"react"` avec `import type`
- **Champ contrôlé** – `value` + `onChange`, l'état est la source de vérité
- **Valeurs** – `e.target.value` est toujours une chaîne, `e.target.checked` pour les cases à cocher
- **Soumission** – `onSubmit` sur le formulaire et `preventDefault()`
- **État objet** – Un objet pour tout le formulaire, mis à jour avec spread et nom de propriété calculé
- **Validation** – Erreurs déduites de l'état, affichées après la première soumission

Prochaine séance : Séance 13 – Collection comme état

Exercice filé S12

1. Créez un composant `RecipeForm` avec un état `form` contenant : titre, description, URL de l'image, temps de préparation, temps de cuisson, portions, difficulté et catégorie
2. Utilisez des `TextField` MUI ; la catégorie est une liste déroulante générée à partir du module `categories` (avec `map`)
3. Pour la difficulté, utilisez un `Rating` modifiable (`value` et `onChange` : consultez sa documentation ; attention, la nouvelle valeur est de type `number | undefined`)
4. À la soumission, construisez un objet dont les champs numériques sont convertis en nombres et affichez-le dans la console, puis videz le formulaire
5. Validez : titre d'au moins 3 caractères, temps positifs ou nuls (une recette sans cuisson a un temps de cuisson de 0), au moins 1 portion, catégorie choisie ; les erreurs s'affichent sous les champs après la première soumission
6. Affichez `RecipeForm` au-dessus de la liste des recettes

Exercice complémentaire EC06

1. Créez un projet `EC06` avec Vite + React + TypeScript
2. **Inscription** : un formulaire avec email, mot de passe et confirmation du mot de passe
   - Email : doit contenir `@`
   - Mot de passe : au moins 8 caractères, dont un chiffre
   - Confirmation : identique au mot de passe
   - Les erreurs s'affichent en direct sous chaque champ, dès que l'utilisateur l'a modifié
   - Le bouton d'inscription est désactivé tant que le formulaire est invalide
3. **Formulaire à étapes** : un formulaire en 3 étapes (informations générales, préférences, récapitulatif)
   - Un seul état objet pour toutes les données, un état pour l'étape courante
   - Boutons « Précédent » et « Suivant » ; les données saisies sont conservées quand on revient en arrière
   - La dernière étape affiche un récapitulatif de toutes les données et un bouton « Confirmer »